Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И УПРАВЛЕНИЕ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ, ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОМУ ПРОЕКТУ НА ТЕМУ:

Разработка клиент-серверного приложения для рекомендательной системы мероприятий

Студент группы ИУК4-72Б _____ С.А. Тарасова
(подпись, дата) (И.О. Фамилия)

Руководитель курсового проекта _____ У. В. Никитенко
(подпись, дата) (И.О. Фамилия)

Калуга, 2025

Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ
Заведующий кафедрой __**ИУК4**___
_____(_Ю.Е. Гагарин_)
«_05_»____сентября____2025г.

З А Д А Н И Е
на выполнение курсового проекта

по дисциплине ***Компьютерные сети и Интернет-технологии***

Студент группы ***ИУК4-72Б Тарасова Софья Андреевна***
(фамилия, имя, отчество)

Тема курсового проекта ***Разработка клиент-серверного приложения для рекомендательной системы мероприятий***

Направленность КП ***учебный***
Источник тематики ***кафедра ИУК4***

Задание

Провести анализ требований и технологий разработки программного обеспечения.
Выполнить проектирование программного обеспечения.
Осуществить интеграция компонентов программного обеспечения.

Оформление курсового проекта

Расчетно-пояснительная записка на 42 листах формата А4.
Перечень графического материала КП (плакаты, схемы, чертежи и т.п.):
− Интерфейс приложения – 1 лист формата А3;
− Контекстная диаграмма – 1 лист формата А3;
− Диаграмма декомпозиции – 1 лист формата А3.

Дата выдачи задания «_05_» сентября 2025 г.

Руководитель	05.09.2025	У. В. Никитенко
	(подпись, дата)	(И.О. Фамилия)
Студент	05.09.2025	С.А. Тарасова
	(подпись, дата)	(И.О. Фамилия)

Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

КАЛЕНДАРНЫЙ ПЛАН
на выполнение курсового проекта

по дисциплине ***Компьютерные сети и Интернет-технологии***

Студент группы ***ИУК4-72Б Тарасова Софья Андреевна***
(фамилия, имя, отчество)

Тема курсового проекта ***Разработка клиент-серверного приложения для рекомендательной системы мероприятий***

№	Наименование этапов	Сроки выполнения этапов		Отметка о выполнении	
		план	факт	Руководитель	Куратор
1	Задание на выполнение	1-я нед.			
2	Выполнение логического проектирования программного обеспечения	10-я нед.			
3	Выполнение и окончательное оформление графической части и расчетно-пояснительной записки	14-я нед.			
4	Защита	17-я нед.			

Студент _____ 05.09.2025г. Руководитель _____ 05.09.2025г.
(подпись, дата) (подпись, дата)

РЕФЕРАТ

Расчетно-пояснительная записка 42 с., 9 рисунков, 5 таблиц, 17 источников.

Проектирование клиент-серверного приложения рекомендательной системы событий.

Объектом разработки является система, автоматизирующая подбор и предоставление персонализированных рекомендаций событий для пользователей с учетом их интересов и географического положения.

Цель проекта – разработка рекомендательной системы, позволяющей пользователю быстро получать релевантные предложения о событиях и повышать удобство взаимодействия с информацией о мероприятиях.

Поставленные задачи решаются путем проектирования и разработки клиент-серверного взаимодействия клиентского и серверного приложений с использованием выбранных инструментов и технологий, включая REST API, кеширование данных и интеграцию с внешними сервисами (KudaGo, Яндекс.Карты).

СОДЕРЖАНИЕ

РЕФЕРАТ 4

ВВЕДЕНИЕ 7

1. АНАЛИЗ ТРЕБОВАНИЙ И ТЕХНОЛОГИЙ РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ 9

1.1. Основные требования к разрабатываемой системе 9

1.2 Описание системы 11

1.3 Анализ аналогов и прототипов 14

1.4 Перечень задач, подлежащих решению в процессе разработки 16

1.5 Обоснование выбора СУБД 17

1.6 Обоснование выбора языка программирования 18

1.7 Обоснование выбора среды разработки 18

1.8 Обоснование выбора UI-фреймворка 19

1.9 Обоснование выбора фреймворка для создания сервера 19

1.10 Описание реализуемых архитектур 19

2. ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ПРОДУКТА 22

2.1 Общая структура системы 22

2.2 Используемые внешние сервисы и API 23

2.3 Хранение, обработка данных и структура базы данных 23

2.4 Рекомендательная система 27

2.5 Логика взаимодействия компонентов 29

2.6 Масштабируемость и расширяемость 30

3. КОНТРОЛЬ КАЧЕСТВА И ИНТЕГРАЦИЯ КОМПОНЕНТОВ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ 32

3.1 Тестирование системы 32

3.2 Руководство администратора 32

3.3 Руководство пользователя 34

ЗАКЛЮЧЕНИЕ 40

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 41

ВВЕДЕНИЕ

В условиях цифровизации и роста объёмов информации пользователи все чаще сталкиваются с проблемой избыточного выбора. Это особенно актуально для сферы культурных и развлекательных мероприятий, где ежедневно публикуются сотни событий различной направленности. Отсутствие персонализированного подхода приводит к снижению вовлеченности пользователей и затрудняет поиск действительно интересных мероприятий. В связи с этим разработка рекомендательных систем, способных учитывать предпочтения пользователей и предлагать релевантные события, является актуальной задачей современной информационной среды.

Рекомендательные системы широко применяются в электронной коммерции, потоковых сервисах и социальных сетях, однако в области событийных сервисов они зачастую реализованы в упрощённом виде и не обеспечивают достаточной степени персонализации. Это обусловливает необходимость разработки программного продукта, ориентированного на анализ пользовательского интереса и формирование рекомендаций без жесткой привязки к конкретным датам посещения мероприятий.

Целью выпускной квалификационной работы является разработка клиент-серверного программного продукта — рекомендательной системы событий, обеспечивающей персонализированный подбор мероприятий на основе пользовательских предпочтений и данных о событиях.

Для достижения поставленной цели в работе необходимо решить следующие задачи:

1. Проанализировать предметную область и существующие аналоги рекомендательных систем событий;
2. Сформировать требования к разрабатываемой системе;
3. Спроектировать архитектуру клиент-серверного приложения;
4. Разработать структуру базы данных для хранения информации о событиях и пользователях;

5. Реализовать алгоритмы рекомендаций на основе анализа интересов пользователей;
6. Разработать серверную и клиентскую части приложения;
7. Провести тестирование разработанной системы и оценить её работоспособность.

Объектом исследования в данной работе являются информационные системы рекомендаций.

Предметом исследования являются методы и средства разработки рекомендательной системы событий в рамках клиент-серверной архитектуры.

В процессе выполнения работы были использованы методы системного анализа, объектно-ориентированного проектирования, проектирования баз данных, а также методы разработки веб-приложений [5]. При реализации рекомендательной части применялись алгоритмы анализа пользовательских предпочтений.

1. АНАЛИЗ ТРЕБОВАНИЙ И ТЕХНОЛОГИЙ РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

1.1. Основные требования к разрабатываемой системе

Описание предметной области

Предметной областью разрабатываемой системы является автоматизированный подбор и рекомендация событий культурно-досугового характера для пользователей. К таким событиям относятся концерты, выставки, театральные постановки, лекции, фестивали и иные публичные мероприятия, информация о которых публикуется в открытых источниках и специализированных сервисах.

Современные сервисы событий предоставляют пользователям каталоги мероприятий с возможностью фильтрации по дате, месту проведения и тематике. Однако при большом количестве доступных событий пользователю становится затруднительно самостоятельно находить наиболее интересные варианты. В этой связи возникает необходимость в программных системах, способных анализировать предпочтения пользователя и формировать персонализированные рекомендации.

Разрабатываемая система ориентирована на сценарий, при котором пользователь выражает интерес к событиям, не принимая на себя обязательств по посещению в конкретную дату. Это позволяет формировать рекомендации на основе общего профиля интересов, а не фиксированных планов.

Текущее состояние предметной области и существующие проблемы

В настоящее время в области событийных информационных систем широко распространены агрегаторы мероприятий, предоставляющие обширные базы данных событий. Основными проблемами таких систем являются:

— отсутствие или слабая персонализация рекомендаций;

— ориентация преимущественно на фильтрацию, а не на интеллектуальный подбор контента;

— перегруженность интерфейсов при большом количестве доступных событий;

— ограниченное использование данных о поведении пользователя для формирования рекомендаций.

Во многих существующих сервисах рекомендации формируются на основе популярности событий или редакторских подборок, что не всегда соответствует индивидуальным интересам пользователя. Кроме того, некоторые системы требуют явного указания дат посещения, что снижает гибкость взаимодействия с пользователем.

В результате пользователь вынужден самостоятельно просматривать большое количество событий, что снижает удобство использования сервиса и эффективность поиска релевантной информации.

Актуальные тенденции в предметной области

Одной из ключевых тенденций развития событийных информационных систем является внедрение рекомендательных алгоритмов, ориентированных на персонализацию пользовательского опыта. Современные системы стремятся учитывать историю взаимодействия пользователя с контентом, его интересы и предпочтения, а также контекст использования сервиса.

Также актуальным направлением является развитие клиент-серверных веб-приложений, обеспечивающих доступ к функциональности системы с различных устройств без необходимости установки дополнительного программного обеспечения. Использование REST-ориентированных API и современных веб-фреймворков позволяет создавать масштабируемые и расширяемые системы.

Дополнительно наблюдается тенденция к упрощению пользовательских сценариев: отказ от избыточных обязательств со стороны пользователя, минимизация вводимых данных и фокус на анализе косвенного интереса. В рамках данной курсовой работы эти тенденции учитываются при проектировании и реализации рекомендательной системы событий.

1.2 Описание системы

Наименование системы

Настоящее Техническое задание определяет требования и порядок создания «Клиент-серверного приложения для рекомендательной системы мероприятий» [8].

Исполнитель

Исполнителем проекта является студент Калужского филиала МГТУ им. Н. Э. Баумана, факультета ИУК, группы ИУК4-72Б, Тарасова Софья Андреевна.

Краткая характеристика области применения

Разрабатываемая система предназначена для использования в области информационных сервисов, связанных с подбором и рекомендацией культурно-досуговых мероприятий. Система может применяться в веб-среде для предоставления пользователям персонализированных рекомендаций событий на основе их интересов и истории взаимодействия с приложением.

Область применения системы включает онлайн-платформы и сервисы, ориентированные на поиск и анализ событий, а также информационные ресурсы, предоставляющие пользователям доступ к актуальной базе мероприятий.

Целевая аудитория

Целевой аудиторией системы являются пользователи, заинтересованные в посещении культурных и развлекательных мероприятий, но испытывающие трудности с выбором подходящих событий из большого объема доступной информации. К таким пользователям относятся студенты, молодые специалисты и активные пользователи интернета, использующие веб-приложения для поиска информации и планирования досуга.

Также системой могут пользоваться администраторы, ответственные за сопровождение и обновление базы данных событий.

Назначение системы

Назначением разрабатываемой системы является автоматизированный подбор и рекомендация событий для пользователей с учетом их интересов и

предпочтений. Система предназначена для упрощения процесса поиска релевантных мероприятий и повышения удобства взаимодействия пользователя с событийным контентом.

Цели создания системы

Основной целью создания системы является разработка программного продукта, обеспечивающего персонализированный подбор событий на основе анализа пользовательских предпочтений. Дополнительными целями являются повышение удобства пользовательского интерфейса, снижение времени поиска интересных мероприятий и создание гибкой архитектуры, допускающей дальнейшее расширение функциональности.

Требования к приложению

Разрабатываемое приложение должно обеспечивать:

— доступ к базе данных событий через веб-интерфейс;

— возможность просмотра списка событий и получения рекомендаций;

— корректную обработку пользовательских запросов;

— стабильную работу при одновременной работе нескольких пользователей;

— возможность масштабирования и модификации функционала.

Требования к клиентской части системы

Клиентская часть системы должна:

— предоставлять удобный и интуитивно понятный пользовательский интерфейс;

— обеспечивать отображение списка событий и рекомендованных мероприятий;

— взаимодействовать с серверной частью посредством HTTP-запросов;

— корректно работать в современных веб-браузерах;

— не требовать установки дополнительного программного обеспечения.

Требования к серверной части системы

Серверная часть системы должна:

— обеспечивать обработку запросов от клиентской части;

— реализовывать бизнес-логику приложения;

— взаимодействовать с базой данных для хранения и получения информации о событиях и пользователях;

— реализовывать алгоритмы рекомендаций;

— обеспечивать защиту и целостность данных.

Требования к надежности

Система должна обеспечивать устойчивую работу при корректных условиях эксплуатации. В случае возникновения ошибок система должна корректно обрабатывать исключительные ситуации без потери данных и с минимальным влиянием на пользовательский опыт.

Время восстановления после отказа

Время восстановления работоспособности системы после отказа не должно превышать времени, необходимого для перезапуска серверного приложения и восстановления соединения с базой данных. В рамках курсового проекта предполагается восстановление работоспособности в течение нескольких минут.

Условия эксплуатации

Эксплуатация системы предполагается в условиях стандартной серверной инфраструктуры с доступом к сети Интернет. Пользовательская часть системы эксплуатируется на устройствах с современными веб-браузерами. Специальные требования к аппаратному обеспечению не предъявляются.

Требования к квалификации и численности персонала

Для эксплуатации системы со стороны пользователя не требуется специальных навыков, достаточно базового уровня компьютерной грамотности. Обслуживание серверной части и базы данных может осуществляться одним специалистом, обладающим знаниями в области администрирования веб-приложений и баз данных.

Требования к защите информации и программ

Система должна обеспечивать базовую защиту информации, включая разграничение прав доступа пользователей и защиту серверной части от

несанкционированного доступа. Передача данных между клиентской и серверной частями должна осуществляться с использованием стандартных протоколов веб-взаимодействия.

Этапы разработки

Разработка системы осуществляется поэтапно и включает следующие этапы:

— анализ предметной области и формирование требований к системе;
— проектирование архитектуры приложения и структуры базы данных;
— реализация серверной и клиентской частей системы;
— интеграция компонентов и реализация алгоритмов рекомендаций;
— тестирование системы и исправление выявленных ошибок;
— подготовка документации по использованию системы.

1.3 Анализ аналогов и прототипов

В настоящее время существует ряд онлайн-сервисов, предоставляющих пользователям информацию о культурных и развлекательных мероприятиях. Наиболее известными из них являются сервисы KudaGo, Афиша и Eventbrite. Эти платформы используются в качестве аналогов при анализе функциональных и пользовательских характеристик разрабатываемой системы.

Сервис KudaGo ориентирован на подбор событий в крупных городах и предоставляет пользователю каталог мероприятий с возможностью фильтрации по дате, месту проведения и тематике. Основным преимуществом данного сервиса является широкий охват событий и удобная система категорий. Однако рекомендации в основном носят общий характер и формируются преимущественно на основе популярности мероприятий, без учета индивидуальных интересов пользователя. Персонализация в сервисе реализована ограниченно и не является ключевым элементом взаимодействия.

Сервис Афиша также предоставляет обширный список мероприятий, включая киносеансы, концерты и выставки. К сильным сторонам данного сервиса относится узнаваемость платформы и наличие дополнительного

контента, такого как обзоры и рецензии. В то же время интерфейс сервиса может быть перегружен информацией, а рекомендации зачастую основаны на редакционных подборках, что снижает их индивидуальную релевантность для конкретного пользователя.

Eventbrite представляет собой международную платформу для поиска и организации мероприятий. Сервис отличается развитой инфраструктурой, возможностью регистрации и покупки билетов, а также масштабируемостью. Однако ориентация Eventbrite в большей степени направлена на организаторов мероприятий и коммерческие события. Персонализация рекомендаций для конечного пользователя выражена слабо, а интерфейс может быть избыточно сложным для быстрого поиска интересных мероприятий.

Анализ существующих аналогов показывает, что большинство сервисов сосредоточены на предоставлении большого объема информации и инструментов фильтрации, но уделяют недостаточное внимание интеллектуальной персонализации контента. Пользователь часто вынужден самостоятельно просматривать большое количество событий, чтобы найти действительно интересные варианты.

Разрабатываемая система отличается акцентом на персонализированный подбор событий на основе интересов пользователя и истории его взаимодействия с приложением. В отличие от рассмотренных аналогов, система не требует от пользователя обязательного указания конкретных дат посещения мероприятий, что повышает гибкость использования сервиса. Кроме того, интерфейс системы ориентирован на простоту и минимализм, что снижает когнитивную нагрузку и ускоряет процесс поиска рекомендаций.

Таким образом, разрабатываемая система сочетает в себе функциональность событийных сервисов и элементы рекомендательных систем, делая упор на индивидуальные предпочтения пользователя и удобство взаимодействия, что позволяет выделить её среди существующих аналогов.

1.4 Перечень задач, подлежащих решению в процессе разработки

Для достижения цели курсового проекта в процессе разработки программной системы необходимо решить ряд взаимосвязанных задач, охватывающих этапы анализа, проектирования, реализации и тестирования программного продукта.

Одной из ключевых задач является организация сбора и хранения данных о событиях. В рамках данной задачи требуется обеспечить получение информации о мероприятиях, включая их основные характеристики, такие как название, описание, место проведения, даты, категории и теги. Также необходимо спроектировать структуру базы данных, обеспечивающую целостность, удобство хранения и последующего использования данных в процессе формирования рекомендаций.

Важной задачей является разработка модели рекомендаций. Для формирования персонализированных предложений в системе используется подход, основанный на анализе содержания событий и пользовательских интересов. В качестве основного метода применяется вычисление косинусного сходства (cosine similarity) между векторами признаков событий и профиля пользователя.

Модель рекомендаций учитывает историю взаимодействия пользователя с системой, включая отмеченные или просмотренные события, на основе которых формируется вектор пользовательских предпочтений. Для повышения релевантности рекомендаций дополнительно используется географический фактор, позволяющий учитывать расстояние между пользователем и местом проведения события. Это позволяет отдавать приоритет событиям, находящимся в географически удобной доступности для пользователя.

Реализация данной модели обеспечивает простоту, интерпретируемость результатов и возможность дальнейшего расширения, например за счёт добавления новых факторов оценки событий без существенного изменения архитектуры системы.

Следующей задачей является реализация клиент-серверного взаимодействия. В рамках данной задачи требуется разработать серверную часть приложения, обеспечивающую обработку запросов от клиента, доступ к базе данных и выполнение рекомендательных алгоритмов. Клиентская часть должна обеспечивать удобный пользовательский интерфейс и взаимодействовать с сервером посредством стандартных веб-протоколов.

Завершающей задачей является проведение тестирования и подготовка документации. Тестирование направлено на проверку корректности работы основных функций системы, устойчивости приложения к ошибкам и соответствия реализованного функционала поставленным требованиям. Документация должна содержать описание принципов работы системы, а также рекомендации по использованию и сопровождению программного продукта.

1.5 Обоснование выбора СУБД

В качестве системы управления базами данных для разрабатываемого приложения была выбрана СУБД PostgreSQL. Данная СУБД является объектно-реляционной и широко применяется при разработке серверных веб-приложений.

Выбор PostgreSQL обусловлен её надежностью, высокой производительностью и поддержкой сложных типов данных. В частности, PostgreSQL предоставляет возможность работы с массивами и JSON-структурами, что удобно при хранении информации о событиях, тегах и пользовательских предпочтениях. Кроме того, СУБД обеспечивает поддержку транзакций и механизмов обеспечения целостности данных, что является важным требованием для корректной работы системы.

PostgreSQL является свободно распространяемой СУБД, поддерживает масштабирование и хорошо интегрируется с языком программирования Python, что делает её подходящим решением для реализации серверной части рекомендательной системы.

1.6 Обоснование выбора языка программирования

В качестве основного языка программирования для разработки серверной части системы был выбран язык Python. Данный язык широко используется при создании веб-приложений и рекомендательных систем.

Основными преимуществами Python являются простота синтаксиса, высокая читаемость кода и наличие большого количества библиотек и фреймворков. Для разработки веб-приложений Python предоставляет развитую экосистему инструментов, а для реализации алгоритмов машинного обучения и анализа данных — библиотеки для работы с числовыми данными и моделями рекомендаций.

Использование Python позволяет эффективно реализовать как серверную логику приложения, так и рекомендательный компонент в рамках единого программного стека, что упрощает разработку и сопровождение системы.

1.7 Обоснование выбора среды разработки

Для разработки программного продукта использовались интегрированные среды разработки PyCharm и Visual Studio Code. Данные среды широко применяются при разработке приложений на языке Python и обеспечивают необходимый набор инструментов для эффективной работы.

PyCharm предоставляет расширенные возможности для анализа кода, отладки и управления зависимостями проекта. Visual Studio Code отличается гибкостью и легковесностью, а также поддерживает большое количество расширений для работы с Python, веб-технологиями и системами контроля версий.

Обе среды разработки обеспечивают интеграцию с системой контроля версий Git, что позволяет отслеживать изменения в коде и упрощает командную и индивидуальную разработку.

1.8 Обоснование выбора UI-фреймворка

Для разработки пользовательского интерфейса приложения был выбран UI-фреймворк Bootstrap. Данный фреймворк предназначен для создания адаптивных и кроссплатформенных веб-интерфейсов.

Bootstrap предоставляет готовые компоненты интерфейса, такие как формы, кнопки, таблицы и навигационные элементы, что позволяет ускорить процесс разработки клиентской части приложения. Использование Bootstrap обеспечивает корректное отображение интерфейса на различных устройствах и в разных браузерах без необходимости разработки сложных пользовательских стилей.

Выбор Bootstrap обусловлен его простотой, широким распространением и хорошей совместимостью с серверными веб-фреймворками.

1.9 Обоснование выбора фреймворка для создания сервера

В качестве фреймворка для разработки серверной части приложения был выбран Flask. Flask является легковесным веб-фреймворком на языке Python, предназначенным для создания веб-приложений и REST-ориентированных сервисов.

Выбор Flask обусловлен его простотой и гибкостью, а также отсутствием жестко заданной структуры проекта. Это позволяет реализовать только необходимую функциональность и легко интегрировать серверную часть с базой данных и рекомендательным модулем.

Flask обеспечивает удобную маршрутизацию запросов, поддержку шаблонов и возможность расширения функциональности за счет подключаемых модулей, что делает его подходящим решением для реализации серверной части рекомендательной системы событий.

1.10 Описание реализуемых архитектур

В разрабатываемой системе используется клиент-серверная архитектура, при которой функциональность приложения распределена между клиентской и

серверной частями. Данный подход является стандартным для веб-приложений и обеспечивает удобство масштабирования, сопровождения и развития системы.

Клиентская часть системы отвечает за взаимодействие с пользователем и представление данных. Она формирует запросы к серверу, передаёт параметры поиска и предпочтения пользователя, а также отображает полученные результаты, включая список событий и рекомендации.

Серверная часть системы выполняет обработку запросов от клиента, реализует бизнес-логику приложения и обеспечивает доступ к базе данных. На сервере осуществляется формирование рекомендаций на основе анализа пользовательских интересов с использованием алгоритма косинусного сходства, а также с учетом географического фактора. Результаты обработки возвращаются клиентской части в виде структурированных данных.

Взаимодействие между клиентской и серверной частями осуществляется по протоколу HTTP с использованием REST-ориентированного подхода. Такой способ взаимодействия обеспечивает независимость компонентов системы и позволяет при необходимости изменять или расширять клиентскую часть без модификации серверной логики.

Дополнительно в рамках серверной части можно выделить логическое разделение на уровни обработки запросов, работы с данными и формирования рекомендаций. Такое разделение повышает читаемость кода, упрощает сопровождение системы и создает основу для дальнейшего расширения функциональности.

Выбор клиент-серверной архитектуры позволяет обеспечить гибкость системы, централизованное хранение данных и эффективную реализацию рекомендательной логики, что соответствует требованиям курсового проекта.

Выводы

В первой главе курсового проекта была рассмотрена предметная область разработки рекомендательной системы событий и проанализированы основные требования к разрабатываемому программному продукту. Было показано, что в

условиях большого объёма событийной информации актуальной является задача автоматизированного и персонализированного подбора мероприятий для пользователей.

В ходе анализа были рассмотрены существующие сервисы рекомендаций событий, такие как KudaGo, Афиша и Eventbrite. Анализ аналогов позволил выявить их сильные и слабые стороны, а также обосновать необходимость разработки собственной системы, ориентированной на простоту использования, персонализацию и учет пользовательских предпочтений без жесткой привязки к конкретным датам.

В главе были сформулированы цели и задачи разработки, определены требования к функциональности, надежности и условиям эксплуатации системы. Также было обосновано использование выбранных технологий, включая СУБД PostgreSQL, язык программирования Python, фреймворк Flask для серверной части и Bootstrap для клиентского интерфейса. Рассмотрена клиент-серверная архитектура системы и принципы взаимодействия между её компонентами.

Результаты первой главы создают теоретическую и техническую основу для дальнейшего проектирования программного продукта. Полученные выводы используются при разработке структуры системы, проектировании базы данных и реализации функциональных компонентов, что будет рассмотрено в последующих главах курсового проекта.

2. ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ПРОДУКТА

2.1 Общая структура системы

Разрабатываемая рекомендательная система событий построена по принципу клиент-серверной архитектуры. Такое распределение функций позволяет разделить задачи взаимодействия с пользователем и обработку данных, обеспечивая гибкость, масштабируемость и удобство сопровождения приложения [1, 4].

Клиентская часть отвечает за взаимодействие с пользователем и представление данных. Она обеспечивает отображение списка событий, работу с геопрофилями и картами, отправку запросов к серверу и получение результатов обработки. Клиентская часть формирует запросы к серверу с указанием параметров пользователя и предпочтений, а также отображает рекомендации в удобном для пользователя виде.

Серверная часть выполняет обработку запросов, реализует бизнес-логику системы и обеспечивает доступ к базе данных. На сервере формируются рекомендации на основе алгоритма косинусного сходства с учетом геопрофиля пользователя, обрабатываются данные о событиях и пользователях, а также взаимодействие с внешними API. Сервер управляет кешированием данных и хранением истории взаимодействий пользователей с событиями.

Взаимодействие между клиентской и серверной частями осуществляется через REST API [2, 3], что позволяет поддерживать стандартизированный обмен данными в формате JSON. Такой подход обеспечивает независимость компонентов системы, упрощает разработку и тестирование, а также позволяет при необходимости расширять функциональность как клиентской, так и серверной части без изменения других компонентов приложения.

Использование клиент-серверной архитектуры обеспечивает централизованное хранение данных, эффективное формирование персонализированных рекомендаций и возможность масштабирования приложения при увеличении числа пользователей [6].

2.2 Используемые внешние сервисы и API

Для функционирования разрабатываемой рекомендательной системы используются внешние сервисы и API, которые обеспечивают получение актуальной информации о событиях и работу с географическими данными [7].

KudaGo API используется в качестве основного источника информации о событиях [13]. Через данный API система получает данные о мероприятиях, включая их название, описание, место и дату проведения и теги. Использование KudaGo API позволяет поддерживать актуальность информации в системе без необходимости ручного ввода событий и обеспечивает интеграцию с большим количеством источников культурно-досуговой информации.

Яндекс.Карты API применяется для работы с географическими данными [14]. В частности, API используется для обратного геокодирования (reverse geocode), что позволяет определять координаты мест проведения событий и геопрофиля пользователя по адресу. Также Яндекс.Карты обеспечивают визуализацию местоположения событий на карте и формирование геопрофилей пользователей, что является важным для учета географического фактора при формировании рекомендаций.

Использование этих внешних сервисов позволяет системе эффективно интегрировать данные из разных источников, обеспечивать актуальность информации о событиях и учитывать географический контекст при персонализированном подборе мероприятий для пользователей.

2.3 Хранение, обработка данных и структура базы данных

Для функционирования разрабатываемой системы используется реляционная база данных PostgreSQL [16]. Реляционная структура обеспечивает быстрый доступ к данным, целостность информации и удобство формирования рекомендаций на основе пользовательских предпочтений и истории взаимодействий.

Основные таблицы базы данных

1. Таблица tags — исходный набор тегов для выбора интересов пользователем:

Поле	Тип	Описание
id	INT, PK	Идентификатор тега
name	VARCHAR(255)	Название тега

Табл.1 Таблица tags

После первоначального выбора интересов пользователем данные таблицы в дальнейшем не используются напрямую.

2. Таблица users — информация о пользователях:

Поле	Тип	Описание
id	SERIAL, PK	Идентификатор пользователя
username	TEXT, UNIQUE, NOT NULL	Имя пользователя
password_hash	TEXT, NOT NULL	Хэш пароля
interests	JSONB, DEFAULT '[]'	Список интересов пользователя
geoprofiles	JSONB, DEFAULT '[]'	Геопрофили пользователя (координаты и название города)
active_profile_id	TEXT	Активный профиль предпочтений пользователя

Табл.2 Таблица users

3. Таблица events_cache — кешированные данные о событиях, используемые для рекомендаций:

Поле	Тип	Описание
id	INT, PK	Идентификатор события
title	TEXT	Название события
place	JSONB	Место проведения события (город, координаты, название)

start_date	TIMESTAMP	Ближайшая актуальная дата начала
tags	TEXT[]	Теги события
thumbnail	TEXT	Миниатюра события

Табл.3 Таблица events_cache

Кеш обновляется периодически скриптом, который загружает события, опубликованные с даты последнего обновления, заменяя существующие записи по идентификатору.

4. Таблица events_archive — архив всех событий, включая прошедшие:

Поле	Тип	Описание
id	INT, PK	Идентификатор события
title	TEXT	Название события
place	JSONB	Место проведения события
dates	JSONB	Все даты события (включая прошедшие)
tags	TEXT[]	Теги события
thumbnail	TEXT	Миниатюра события

Табл.4 Таблица events_archive

Архив используется для формирования списка отслеживаемых пользователем событий и сохранения истории взаимодействий, даже если событие уже прошло.

5. Таблица tracked_events — связь пользователей и отслеживаемых событий:

Поле	Тип	Описание
id	SERIAL, PK	Идентификатор записи
user_id	INTEGER, FK → users(id)	Пользователь, отслеживающий событие
event_id	INTEGER, FK → events_archive(id)	Отслеживаемое событие

Табл.5 Таблица tracked_events

Таблица обеспечивает уникальность каждой пары пользователь-событие и позволяет учитывать события, которые пользователь отметил, включая уже прошедшие.

Логика обработки данных

В разрабатываемой системе активно используется кеширование данных о событиях. Таблица events_cache хранит актуальные события и служит основным источником данных при формировании рекомендаций. Это позволяет снизить нагрузку на внешний API KudaGo, поскольку при каждом запросе пользователя не требуется повторно обращаться к сервису [10]. Кеш обновляется периодически скриптом, который загружает события, опубликованные с момента последнего обновления, заменяя существующие записи по идентификатору события. Такой подход обеспечивает высокую производительность системы и минимизирует задержки при выдаче рекомендаций пользователю.

Обоснование структуры

Для хранения информации о пользователях и событиях в таблицах используется денормализованная структура с JSONB-полями. Например, геопрофили пользователя и место проведения события хранятся в виде JSONB-объектов, а список интересов пользователя также представлен JSONB. Такой подход позволяет:

— уменьшить количество соединений (JOIN) при формировании рекомендаций, что критично для быстрого вычисления сходства событий с профилем пользователя;

— гибко хранить сложные и разнородные данные, которые могут включать координаты, имена городов, списки тегов и т.д.;

— легко расширять структуру без необходимости создания множества вспомогательных таблиц при добавлении новых атрибутов.

Таким образом, сочетание кеширования и денормализованной структуры обеспечивает эффективность работы системы, ускоряет формирование персонализированных рекомендаций и упрощает обработку сложных данных о событиях и пользователях. Архивная таблица events_archive вместе с таблицей tracked_events дополнительно сохраняет историю взаимодействий пользователя с событиями, включая прошедшие, что позволяет системе корректно учитывать предпочтения пользователя даже для завершённых мероприятий.

2.4 Рекомендательная система

В разрабатываемой системе персонализированные рекомендации формируются с использованием алгоритма косинусного сходства (cosine similarity), дополненного учетом географического фактора пользователя и местоположения события [15]. Алгоритм позволяет оценивать релевантность событий на основе интересов пользователя и расстояния до места проведения мероприятия.

Основные шаги формирования рекомендаций

Получение данных пользователя: сервер подключается к базе данных и извлекает информацию о пользователе, включая вектор интересов. Вектор представляет собой словарь, где ключами являются теги, а значениями — текущие коэффициенты интереса пользователя к этим тегам.

Определение текущей геопозиции пользователя: во время выполнения запроса сервер получает географическую позицию и город пользователя.

Получение данных о событии: через API KudaGo извлекаются сведения о событии — город и координаты места проведения, список тегов события.

Расчет коэффициента рекомендации: для каждого события формируется итоговый показатель релевантности по формуле:

$$\text{score} = \beta \cdot \text{simscore} + (1 - \beta) \cdot \text{geoscore},$$

где:

— β = 0.75 (SIMILARITY_CONTRIBUTION) — весовой коэффициент для сходства интересов;

— simscore — косинусное сходство между вектором интересов пользователя и тегами события;

— $\text{geoscore} = \alpha \cdot e^{-d/\tau}$, где

- d — расстояние между пользователем и местом проведения события;
- τ = 50 (GEOSCORE_DESCEND) — коэффициент затухания с расстоянием;
- α=1.0 (SAME_CITY_GRADE), если город пользователя совпадает с городом события, иначе α=0.3 (CITY_CHANGE_PENALTY).

Обновление интересов пользователя

При отслеживании события коэффициенты тегов, присутствующих в событии, увеличиваются на 1 + LEARNING_RATE_WATCH. Если тег отсутствовал в векторе интересов пользователя, он добавляется с начальным значением 0.2 (NEW_TAG_INIT_WATCH). Одновременно коэффициенты тегов, не затронутых событием, уменьшаются на 1 - DECAY_RATE.

При оценке события: если пользователю понравилось событие, соответствующие коэффициенты тегов умножаются на 1 + LEARNING_RATE_GRADE, если не понравилось — на 1 - LEARNING_RATE_GRADE. Новые теги, которые появляются при оценке, инициализируются значением 0.4 (NEW_TAG_INIT_GRADE).

Интеграция с серверной частью

Алгоритм реализован на сервере и вызывается при запросе рекомендаций. Сервер получает информацию о пользователе и событии, рассчитывает коэффициент релевантности, обновляет вектор интересов и возвращает клиенту список персонализированных рекомендаций в порядке убывания их оценочного показателя. Такой подход обеспечивает динамическое формирование рекомендаций с учетом текущих интересов пользователя и его географического

положения, а также постепенное обучение модели на основе действий пользователя.

2.5 Логика взаимодействия компонентов

В разрабатываемой системе взаимодействие между клиентской и серверной частями осуществляется через REST API. Клиент формирует запрос, содержащий параметры пользователя, включая идентификатор, интересы и текущую геопозицию, после чего сервер обрабатывает запрос и возвращает персонализированные рекомендации в виде структурированных данных.

Обработка запроса сервером включает следующие шаги:

1. Получение данных пользователя: сервер извлекает информацию о пользователе из базы данных, включая вектор интересов и геопрофиль.
2. Формирование набора событий для рекомендации: сервер обращается к таблице events_cache, чтобы получить актуальные события. Кеширование позволяет сократить количество обращений к внешнему API KudaGo и ускоряет обработку запроса.
3. Обработка внешних данных при обновлении кеша: периодически сервер запускает скрипт обновления кеша, который получает новые события через KudaGo API и при необходимости данные о местоположении через Яндекс.Карты API. Полученные события заменяют существующие записи в таблице events_cache по идентификатору события.
4. Формирование рекомендаций: сервер применяет алгоритм cosine similarity с учетом геопрофиля пользователя, рассчитывает итоговый коэффициент релевантности каждого события и сортирует их по убыванию.
5. Возврат данных клиенту: сформированный список событий передается клиентской части через REST API, где отображается пользователю в удобном формате.

Таким образом, архитектура системы обеспечивает централизованную обработку данных, эффективное использование кеша для ускорения формирования рекомендаций и минимизацию обращений к внешним сервисам. Такой подход гарантирует быстроту отклика приложения и поддерживает актуальность информации о событиях, одновременно сохраняя гибкость и расширяемость системы [17].

2.6 Масштабируемость и расширяемость

Разрабатываемая рекомендательная система спроектирована с учетом возможности масштабирования и дальнейшего расширения функциональности.

Добавление новых источников данных

Архитектура системы позволяет интегрировать дополнительные внешние API для получения информации о событиях или географических данных. Для этого достаточно реализовать соответствующие скрипты обновления кеша и обеспечить согласованность формата данных с таблицами events_cache и events_archive.

Расширение модели рекомендаций

Рекомендательная система построена на основе алгоритма cosine similarity с учетом геопрофиля пользователя. Структура и логика обработки данных позволяют легко добавлять новые алгоритмы или факторы при формировании рекомендаций. Например, возможно введение дополнительных коэффициентов для учета популярности события, временных предпочтений или интересов социальных групп пользователей.

Масштабирование сервера

Клиент-серверная архитектура обеспечивает возможность горизонтального масштабирования серверной части. При увеличении количества пользователей можно развернуть дополнительные серверные инстансы с общей базой данных и централизованным кешем, что позволит поддерживать высокую производительность и минимальное время отклика при формировании рекомендаций.

Таким образом, архитектура и структура системы обеспечивают не только текущее эффективное функционирование, но и возможность адаптации к росту пользовательской базы, изменению требований и интеграции новых источников данных.

Выводы

Во второй главе была рассмотрена проектная часть разрабатываемой рекомендательной системы событий. Были описаны общая структура системы, используемые внешние сервисы и API, а также организация хранения и обработки данных в базе PostgreSQL с учетом кеширования и архивирования событий. Подробно разобрана структура базы данных, включая таблицы пользователей, кеша событий и архива взаимодействий, что обеспечивает эффективное формирование персонализированных рекомендаций.

Особое внимание было уделено проектированию рекомендательной системы, основанной на алгоритме косинусного сходства и учете геопрофиля пользователя. Были определены методы расчета коэффициента релевантности событий, а также правила обновления интересов пользователя при отслеживании и оценке событий. Также рассмотрена логика взаимодействия компонентов системы, использование кеша и внешних API для оптимизации работы и ускорения формирования рекомендаций.

Наконец, была проанализирована масштабируемость и расширяемость системы, что позволяет интегрировать новые источники данных, модифицировать алгоритмы рекомендаций и увеличивать производительность серверной части при росте числа пользователей.

Таким образом, проектная часть обеспечивает эффективную, гибкую и расширяемую архитектуру системы, готовую к дальнейшей реализации и интеграции с клиентской частью приложения.

3. КОНТРОЛЬ КАЧЕСТВА И ИНТЕГРАЦИЯ КОМПОНЕНТОВ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

3.1 Тестирование системы

Тестирование системы производилось с использованием персонального компьютера под управлением операционной системы Windows. Серверная часть приложения функционировала на веб-фреймворке Flask и обеспечивала обработку HTTP-запросов, поступающих от клиентской части. В качестве системы управления базами данных использовалась PostgreSQL. Для проверки корректности работы серверного API применялся инструмент Postman [9, 12].

Тестирование серверной части включало проверку корректности выполнения запросов к базе данных, устойчивости приложения к ошибочным и некорректным запросам, а также корректности формирования ответов при получении рекомендаций событий. Особое внимание уделялось проверке работы механизмов кеширования данных о событиях и корректности расчёта коэффициентов рекомендаций.

Клиентская часть тестировалась вручную в веб-браузере. Проверялась корректность отображения интерфейса, отправка запросов к серверу, получение и визуализация рекомендованных событий, а также работа функциональности регистрации, авторизации и взаимодействия пользователя с событиями. Тестирование проводилось без использования автоматизированных средств и основывалось на ручной проверке сценариев использования системы.

3.2 Руководство администратора

Данный раздел содержит описание основных действий, необходимых для администрирования и сопровождения рекомендательной системы событий [11].

Запуск системы

Для запуска системы администратору необходимо выполнить следующие действия:

— Убедиться в наличии установленной СУБД PostgreSQL и доступности базы данных, используемой серверной частью приложения.

— Запустить серверное приложение на базе фреймворка Flask. Сервер принимает HTTP-запросы от клиентской части и обрабатывает их в соответствии с реализованной логикой.

— Открыть клиентскую часть приложения в веб-браузере. Взаимодействие клиента с сервером осуществляется через REST API.

Перед началом эксплуатации рекомендуется проверить доступность основных API-эндпоинтов с использованием инструмента Postman.

Обновление кеша событий

Обновление кеша данных о событиях осуществляется с помощью отдельного скрипта, предназначенного для загрузки актуальных событий из внешнего сервиса KudaGo API. Скрипт выполняет следующие операции:

— запрашивает данные о событиях, опубликованных с момента последнего обновления;

— обновляет записи в таблице events_cache, заменяя существующие данные по идентификатору события;

— обеспечивает актуальность информации, используемой рекомендательной системой.

Обновление кеша может выполняться вручную администратором или запускаться периодически по расписанию.

Управление базой данных

Администратор отвечает за контроль состояния базы данных PostgreSQL, включая:

— мониторинг корректности хранения данных пользователей и событий;

— резервное копирование базы данных;

— контроль целостности таблиц users, events_cache, events_archive и tracked_events.

При необходимости администратор может вручную редактировать данные в базе данных с использованием стандартных средств администрирования PostgreSQL.

Мониторинг работы сервера

В процессе эксплуатации системы администратор осуществляет мониторинг работы серверной части, включая:

— контроль запуска и остановки серверного приложения;

— анализ логов сервера для выявления ошибок и исключительных ситуаций;

— проверку корректности обработки пользовательских запросов и формирования рекомендаций.

Таким образом, функции администратора включают обеспечение стабильной работы системы, актуальности данных и корректного взаимодействия всех компонентов программного продукта.

3.3 Руководство пользователя

При входе в систему пользователю предлагается авторизоваться в системе. В случае, если у пользователя нет аккаунта, необходимо зарегистрироваться. В предложенной форме необходимо указать логин и пароль и нажать на кнопку «Зарегистрироваться»

Рис.1 Страница регистрации

После этого пользователю будет предложено выбрать из списка свои интересы для формирования рекомендаций. Необходимо нажать на то, что интересно, и нажать кнопку «Сохранить».

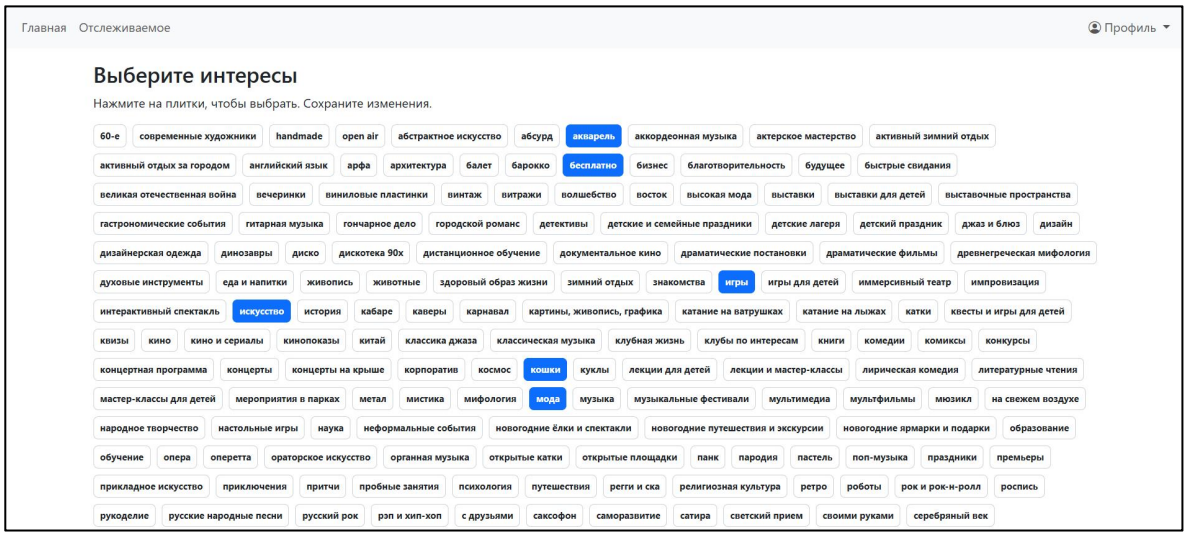

Рис.2 Страница выбора интересов

Если аккаунт есть, то пользователю предлагается войти в аккаунт, используя свой логин и пароль

Рис.3 Страница входа

После авторизации можно перейти на главную, на которой предложены рекомендованные на основе выбранных интересов и текущей геопозиции события. При каждой карточке можно увидеть название мероприятия и место, где оно проходит. Также при наведении на карточку можно увидеть теги метороприятия, совпадающие с интересами пользователя.

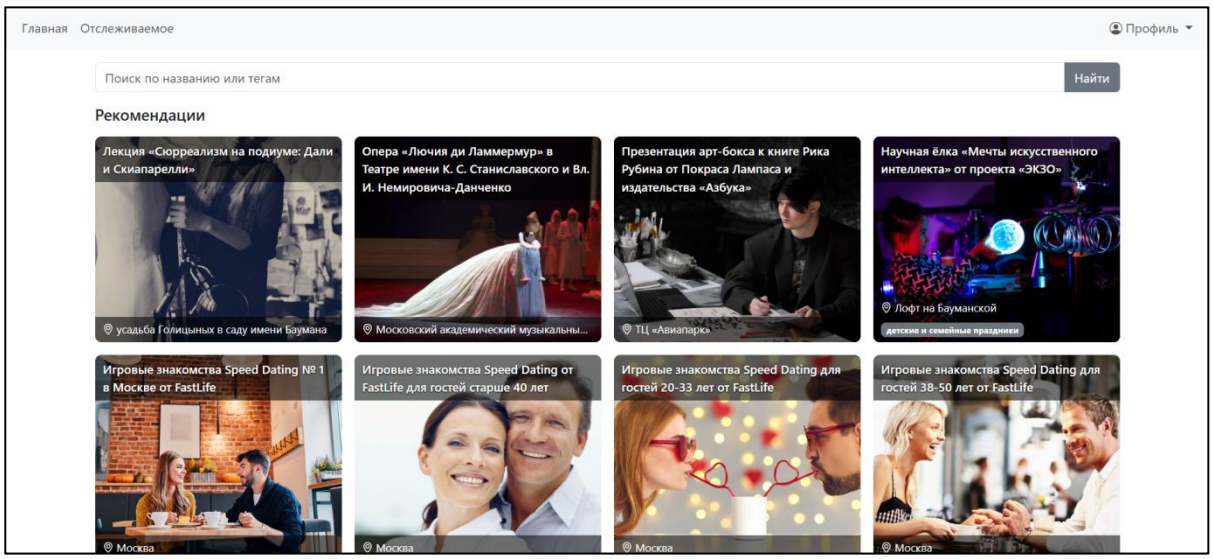

Рис.4 Главная

На каждую карточку можно кликнуть, чтобы просмотреть подробную информацию о мероприятии. В ней можно увидеть описание, актуальные даты проведения, полный список совпадающих интересов, а также местоположение на интерактивной карте при наличии точного адреса.

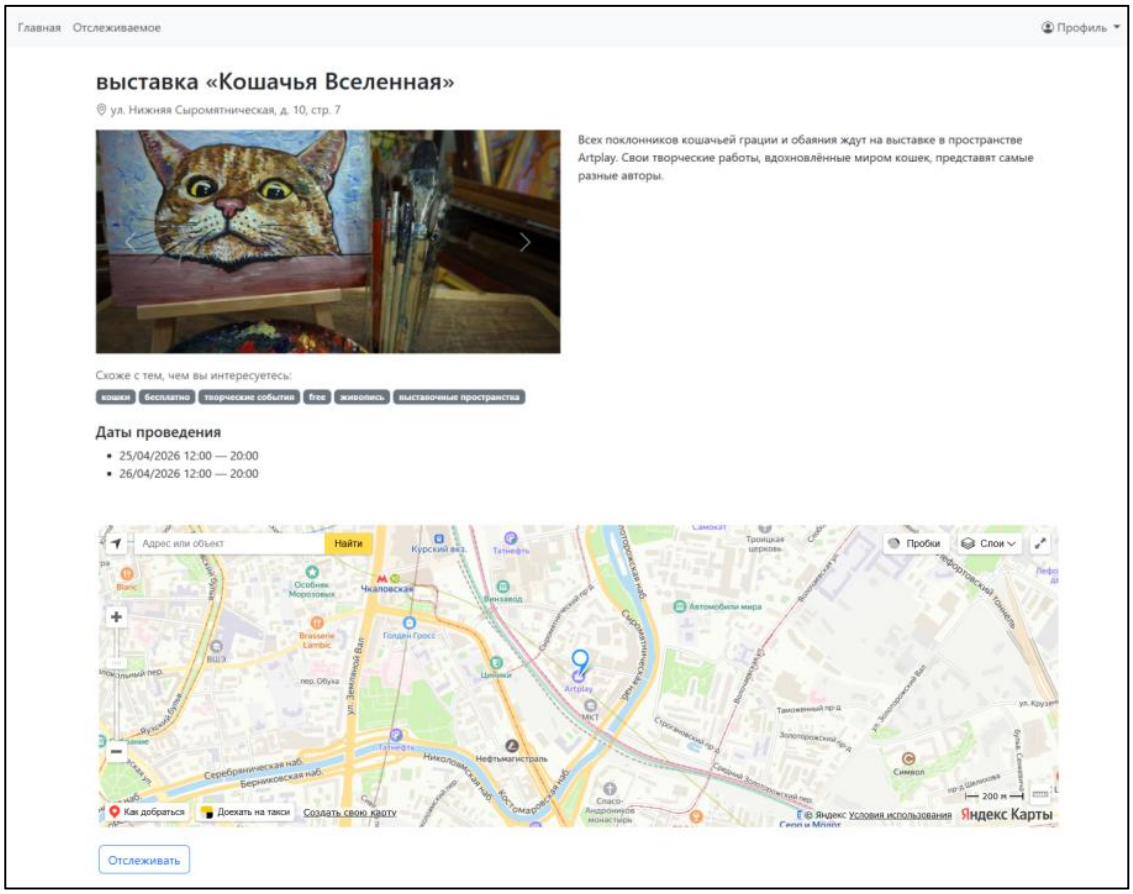

Рис. 5 Детали мероприятия

Каждое событие можно добавить в список отслеживаемых. Получить доступ к списку можно в разделе «Отслеживаемое» на навигационной панели. При нажатии пользователь увидит все отслеживаемые события, которые будут проходить в будущем.

Каждая карточка в этом разделе также является интерактивной - так же, как и на главной можно кликнуть на нее, чтобы увидеть детали события. Для того, чтобы рекомендательная система лучше понимала предпочтения пользователя, рекомендуется размечать посещенные мероприятия кнопками «Понравилось» или «Не понравилось». После этого событие пропадет из отслуживаемого, а мнение о нем будет учтено системой в будущем.

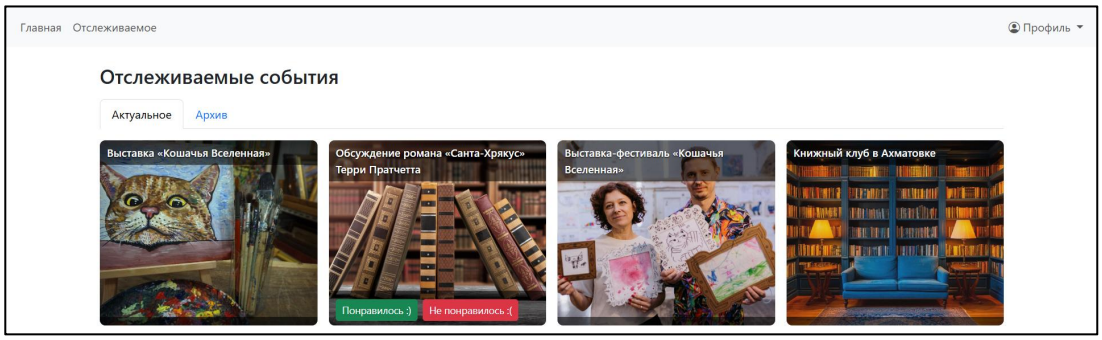

Рис.6 Отслеживаемое - Актуальное

События, дата проведения которых уже позади, существует раздел «Архив». Его структура аналогична разделу «Актуальное».

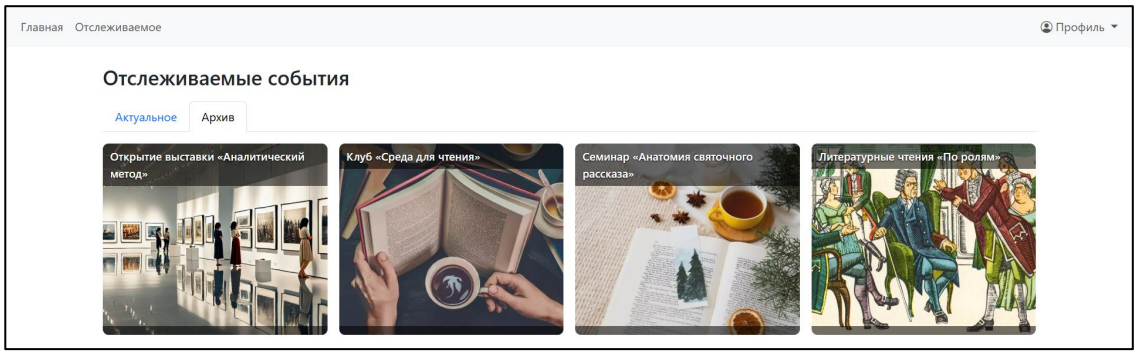

Рис.7 Отслеживаемое - Архив

В рекомендательной системе учтены предпочтения пользователя по местоположению мероприятий. По умолчанию используется текущая геопозиция для построения рекомендаций. В настройках профиля можно создать новые геопрофили, чтобы просматривать рекомендации относительно другой геопозиции.

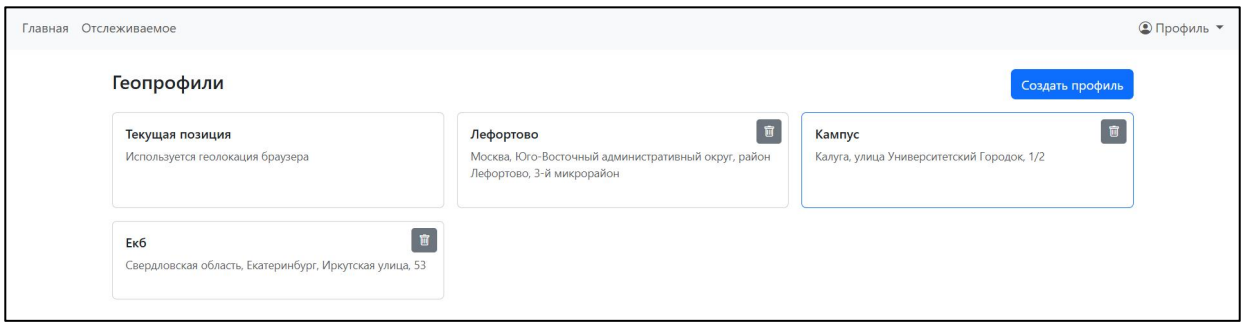

Рис.8 Настройки геопрофилей

Для создания профиля необходимо нажать кнопку «Создать профиль». После этого будет открыто модальное окно в котором нужно заполнить название профиля и выбрать желаемую точку на интерактивной карте. Возможен поиск по адресам.

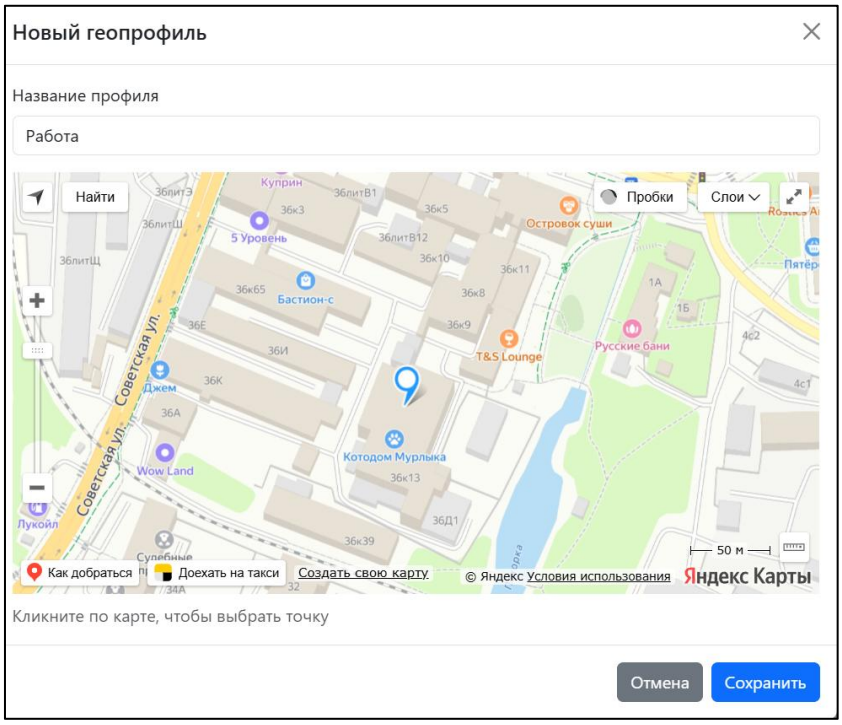

Рис. 9 Форма создания геопрофиля

Возможно переключение между геопрофилями через меню. Для выбора необходимо нажать на предпочитаемый профиль. После этого он будет выделен синей рамкой.

Любой профиль, кроме профиля по умолчанию, можно удалить, нажав на кнопку с иконкой урны в правом верхнем углу карточки геопрофиля.

Выводы

В третьей главе были рассмотрены вопросы контроля качества и интеграции компонентов разработанной рекомендательной системы событий. Описаны условия и методы тестирования серверной и клиентской частей приложения, выполненного преимущественно в ручном режиме с использованием инструмента Postman и проверки пользовательских сценариев в веб-браузере.

В рамках тестирования подтверждена корректность работы серверного API, механизмов взаимодействия с базой данных и кеширования данных о событиях. Проверена устойчивость системы к некорректным запросам, а также корректность формирования и выдачи рекомендаций на основе интересов пользователя и его географического положения.

Также было разработано руководство администратора, описывающее порядок запуска системы, обновления кеша событий, управления базой данных и мониторинга работы серверной части. Наличие пользовательского руководства обеспечивает удобство и доступность системы для конечных пользователей.

Результаты проведённого тестирования показали, что разработанная система функционирует стабильно, соответствует поставленным требованиям и может быть использована для решения задач персонализированного подбора событий.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсового проекта была разработана клиент-серверная рекомендательная система событий, предназначенная для персонализированного подбора мероприятий с учетом интересов и географического положения пользователя. Разработанная система позволяет пользователям получать релевантные рекомендации событий в удобном интерфейсе, а также взаимодействовать с мероприятиями путем их отслеживания и оценки, что повышает удобство поиска и выбора событий.

Система реализует механизм персонализации на основе анализа пользовательских интересов и данных о местоположении, что обеспечивает более точную и релевантную выдачу рекомендаций. Использование кеширования данных о событиях позволяет сократить количество обращений к внешним сервисам и повысить производительность системы. Архитектура приложения обеспечивает гибкость и возможность дальнейшего расширения, включая подключение новых источников данных и развитие модели рекомендаций.

Разработанная система соответствует требованиям технического задания и направлена на оптимизацию процесса поиска и выбора событий, снижая временные затраты пользователя на анализ большого объема информации. Клиент-серверная архитектура и используемые технологии обеспечивают стабильную работу системы и возможность масштабирования при увеличении числа пользователей.

В ходе выполнения курсового проекта были решены все поставленные задачи и достигнута цель разработки рекомендательной системы событий. Кроме того, определены направления для дальнейшего развития и расширения функциональности системы, что подтверждает практическую значимость и перспективность выполненной работы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ачилов, Р. Н. Построение защищенных корпоративных сетей : учебное пособие / Р. Н. Ачилов. — Москва : ДМК Пресс, 2013. — 250 с. — ISBN 978-5-94074-884-7. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/66472

2. Ибе, О. Компьютерные сети и службы удаленного доступа : справочник / О. Ибе. — Москва : ДМК Пресс, 2007. — 336 с. — ISBN 5-94074-080-4. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/1169

3. Семенов, А. А. Сетевые технологии и Интернет : учебное пособие / А. А. Семенов. — Санкт-Петербург : Санкт-Петербургский государственный архитектурно-строительный университет, ЭБС АСВ, 2017. — 148 с. — ISBN 978-5-9227-0662-9. — Текст : электронный // Электронно-библиотечная система IPR BOOKS. — URL: https://www.iprbookshop.ru/66840.html

4. Сергеев, А. Н. Основы локальных компьютерных сетей : учебное пособие для вузов / А. Н. Сергеев. — 3-е изд., стер. — Санкт-Петербург : Лань, 2021. — 184 с. — ISBN 978-5-8114-6855-3. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/152651

5. Сычев, А. В. Web-технологии : учебное пособие / А. В. Сычев. — 3-е изд. — Москва, Саратов : Интернет-Университет Информационных Технологий (ИНТУИТ), Ай Пи Ар Медиа, 2020. — 407 с. — ISBN 978-5-4497-0292-0. — Текст : электронный // Электронно-библиотечная система IPR BOOKS. — URL: https://www.iprbookshop.ru/89412.html

6. Топорков, С. С. Компьютерные сети для продвинутых пользователей : учебное пособие / С. С. Топорков. — Москва : ДМК Пресс, 2009. — 192 с. — ISBN 5-94074-093-6. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/1170

7. ГОСТ 19.103—77. Единая система программной документации. Обозначения программ и программных документов.
8. ГОСТ 19.201—78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению.
9. ГОСТ 19.404—79. Единая система программной документации. Пояснительная записка. Требования к содержанию и оформлению.
10. ГОСТ Р ИСО/МЭК 10746-3—2001. Управление данными и открытая распределённая обработка.
11. ГОСТ Р ИСО/МЭК 15271—02. Процессы жизненного цикла программных средств.
12. ГОСТ Р ИСО/МЭК 15910—2002. Процесс создания документации пользователя программного средства.
13. КудаGo. Документация API сервиса KudaGo [Электронный ресурс]. — Официальная документация сервиса.
14. Яндекс. Документация API Яндекс.Карт [Электронный ресурс]. — Официальная документация Яндекс.
15. Бондаренко М.Ф., Федоров А.А. Рекомендательные системы: методы, алгоритмы, практическое применение. — М.: БХВ-Петербург, 2019.
16. Постгрес Профешнл. PostgreSQL. Руководство разработчика [Электронный ресурс]. — Официальная документация PostgreSQL (русская версия).
17. Хайнеман Дж., Поллис Дж. Алгоритмы и структуры данных. — СПб.: Питер, 2018.